Advanced Computer Architecture for HPC

Prabhsharan Singh
MHPC 25-26

Blocked Matrix Multiplication

Resource: Coka cluster at INFN

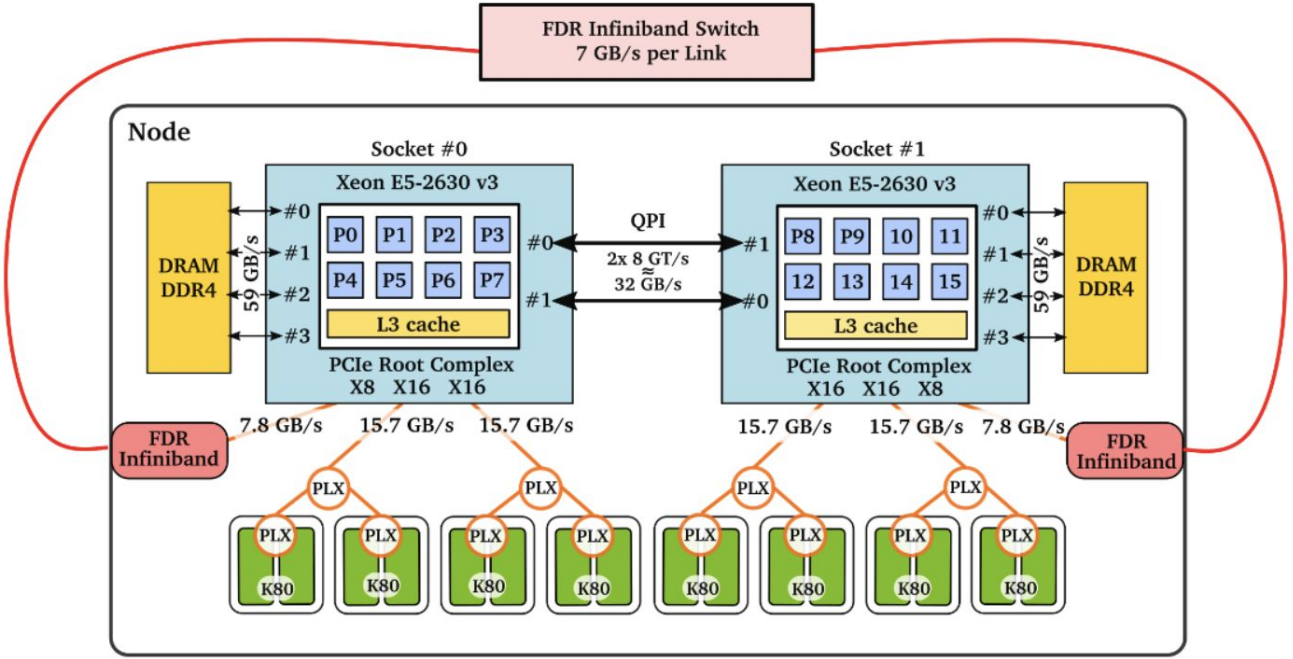


Figure 1: Schematic representation of one of the COKA computing nodes.

Finding the optimal block size for cache-blocking

User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	Performance(MFLOPs)
6221	node01.cluster.taz	2048	98304.00	4	0.38	42105297.00	408.02
6221	node04.cluster.taz	2048	98304.00	8	1.50	35766768.67	480.33
6221	node04.cluster.taz	2048	98304.00	16	6.00	36359278.33	472.50
6221	node04.cluster.taz	2048	98304.00	24	13.50	51186910.33	335.63
6221	node04.cluster.taz	2048	98304.00	32	24.00	51387338.67	334.32

```
double N_kb=N*N*3.0*sizeof(double)/1024.0;
double B_kb=BLOCK_SIZE*BLOCK_SIZE*3.0*sizeof(double)/1024.0;
```

```
L1 Data Cache:
  Total size:              32 KB
  Line size:               64 B
  Number of Lines:        512
  Associativity:            8
```

For N = 2048, B.S = 8 seems optimal

Profile the code by checking performance metrics provided by the PMU

N = 2048, B.S = 16

User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	Performance(MFLOPs)
6221	node04.cluster.taz	2048	98304.00	16	6.00	36346809.67	472.67

```
[psingh@coka logs]$ cat serial_nop_ps_2133639.err

 Performance counter stats for './gemm-x86_64.x 2048 16':

      1,08,997.07 msec task-clock                #    1.000 CPUs utilized
            5,581      context-switches          #    0.051 K/sec
                3      cpu-migrations            #    0.000 K/sec
            2,978      page-faults               #    0.027 K/sec
3,45,41,65,27,256      cycles                    #    3.169 GHz
9,15,39,26,68,202      instructions              #    2.65  insn per cycle
  58,47,90,25,795      branches                  #  536.519 M/sec
     10,90,14,084      branch-misses             #    0.19% of all branches
4,71,48,29,00,642      L1-dcache-loads           # 4325.647 M/sec
  26,64,10,54,286      L1-dcache-load-misses     #    5.65% of all L1-dcache hits
     38,69,97,249      LLC-loads                 #    3.551 M/sec
         1,25,122      LLC-load-misses           #    0.03% of all LL-cache hits

    109.042866984 seconds time elapsed

    108.993517000 seconds user
      0.013996000 seconds sys
```

N = 2048, B.S = 24

User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	Performance(MFLOPs)
6221	node01.cluster.taz	2048	98304.00	24	13.50	51119762.33	336.07

```
[psingh@coka logs]$ cat serial_nop_ps_2133628.err

 Performance counter stats for './gemm-x86_64.x 2048 24':

      1,53,298.09 msec task-clock                #    1.000 CPUs utilized
            7,797      context-switches          #    0.051 K/sec
                2      cpu-migrations            #    0.000 K/sec
            3,367      page-faults               #    0.022 K/sec
 4,87,23,68,28,126      cycles                    #    3.178 GHz
 8,93,97,78,80,138      instructions              #    1.83  insn per cycle
   56,17,41,40,367      branches                  #  366.437 M/sec
      4,98,64,106      branch-misses             #    0.09% of all branches
 4,60,40,30,65,040      L1-dcache-loads           # 3003.319 M/sec
   25,79,17,19,965      L1-dcache-load-misses     #    5.60% of all L1-dcache hits
   23,74,32,33,209      LLC-loads                 #  154.883 M/sec
        1,73,604      LLC-load-misses           #    0.00% of all LL-cache hits

     153.361737229 seconds time elapsed

     153.306732000 seconds user
       0.007997000 seconds sys
```

N = 2048, B.S = 8

User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	Performance(MFLOPs)
6221	node04.cluster.taz	2048	98304.00	8	1.50	35750360.67	480.55

```
[psingh@coka logs]$ cat serial_nop_ps_2133643.err

Performance counter stats for './gemm-x86_64.x 2048 8':

      1,07,152.14 msec task-clock                #    0.999 CPUs utilized
            6,528      context-switches          #    0.061 K/sec
                0      cpu-migrations            #    0.000 K/sec
            3,371      page-faults               #    0.031 K/sec
3,40,28,40,13,328      cycles                    #    3.176 GHz
9,84,07,83,83,996      instructions              #    2.89  insn per cycle
  66,30,74,38,835      branches                  #  618.816 M/sec
     5,33,03,632      branch-misses             #    0.08% of all branches
5,07,09,87,41,470      L1-dcache-loads           # 4732.512 M/sec
  10,49,84,79,316      L1-dcache-load-misses     #    2.07% of all L1-dcache hits
     62,81,48,974      LLC-loads                 #    5.862 M/sec
      3,83,670      LLC-load-misses           #    0.06% of all LL-cache hits

    107.253477950 seconds time elapsed

    107.118747000 seconds user
      0.052986000 seconds sys
```

Use the compiler wisely

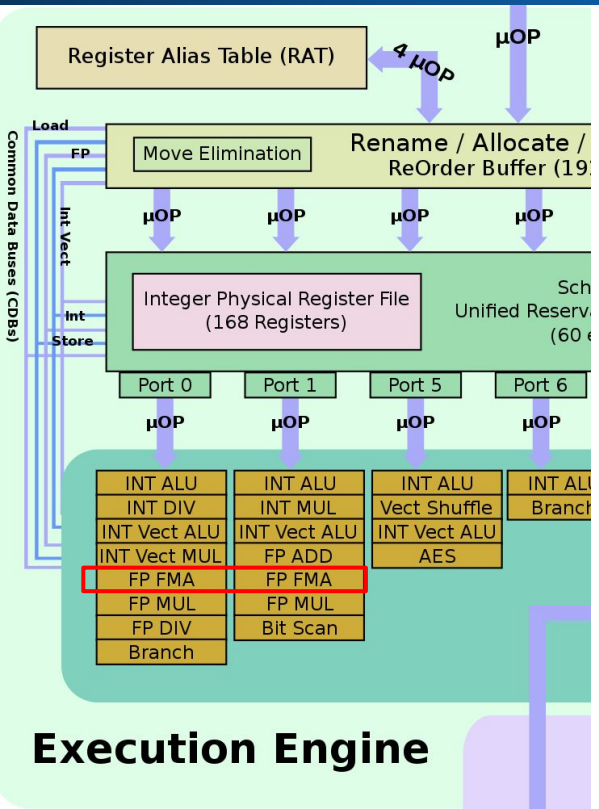
```
gcc -S gemm.c -o gemm_o0.asm
```

```
Shell
.L13:
    movl    -16(%rbp), %eax      ; Load variable i from stack
    imull   -52(%rbp), %eax      ; Calculate i * N
    movl    %eax, %edx
    movl    -36(%rbp), %eax      ; Load variable k from stack
    addl    %edx, %eax           ; Calculate (i * N) + k
    cltq                         ; Convert to 64-bit
    leaq    0(,%rax,8), %rdx     ; Multiply by 8 (sizeof double) to get offset
    movq    -64(%rbp), %rax      ; Load base address of matrix
    addq    %rdx, %rax           ; Add offset to base
    movsd   (%rax), %xmm1        ; FINALLY load the double value
    ... (Repeats entire process for the other matrix) ...
```

```
gcc -O3  -S gemm.c -o gemm_o3.asm
```

```
.L10:
    movsd   (%rax), %xmm0        ; Load value from Matrix A (pointer
                                    in register %rax)
    mulsd   (%rdx), %xmm0        ; Multiply by value from Matrix B (pointer
                                       in %rdx)
    addq    $8, %rax             ; Increment Matrix A pointer by 8 bytes (next
                                         column)
    addq    %rcx, %rdx           ; Increment Matrix B pointer by stride
                                    (next row)
    addsd   %xmm0, %xmm1         ; Accumulate result into sum
    cmpq    %rsi, %rax           ; Check if we are done
    jne     .L10                 ; Loop
```

Utilise your resources wisely



INSTRUCTION SET REFERENCE, V-Z

VFMADD132SD/VFMADD213SD/VFMADD231SD—Fused Multiply-Add of Scalar Double-Precision Floating-Point Values

Opcode/ Instruction	Op/ En	64/32 bit Mode Support	CPUID Feature Flag	Description
VEX.DDS.LIG.66.0F38.W1 99 /r VFMADD132SD xmm1, xmm2, xmm3/m64	RVM	V/V	FMA	Multiply scalar double-precision floating-point value from xmm1 and xmm3/m64, add to xmm2 and put result in xmm1.
VEX.DDS.LIG.66.0F38.W1 A9 /r VFMADD213SD xmm1, xmm2, xmm3/m64	RVM	V/V	FMA	Multiply scalar double-precision floating-point value from xmm1 and xmm2, add to xmm3/m64 and put result in xmm1.
VEX.DDS.LIG.66.0F38.W1 B9 /r VFMADD231SD xmm1, xmm2, xmm3/m64	RVM	V/V	FMA	Multiply scalar double-precision floating-point value from xmm2 and xmm3/m64, add to xmm1 and put result in xmm1.
EVEX.DDS.LIG.66.0F38.W1 99 /r VFMADD132SD xmm1 {k1}{z}, xmm2, xmm3/m64{er}	T1S	V/V	AVX512F	Multiply scalar double-precision floating-point value from xmm1 and xmm3/m64, add to xmm2 and put result in xmm1.
EVEX.DDS.LIG.66.0F38.W1 A9 /r VFMADD213SD xmm1 {k1}{z}, xmm2, xmm3/m64{er}	T1S	V/V	AVX512F	Multiply scalar double-precision floating-point value from xmm1 and xmm2, add to xmm3/m64 and put result in xmm1.
EVEX.DDS.LIG.66.0F38.W1 B9 /r VFMADD231SD xmm1 {k1}{z}, xmm2, xmm3/m64{er}	T1S	V/V	AVX512F	Multiply scalar double-precision floating-point value from xmm2 and xmm3/m64, add to xmm1 and put result in xmm1.

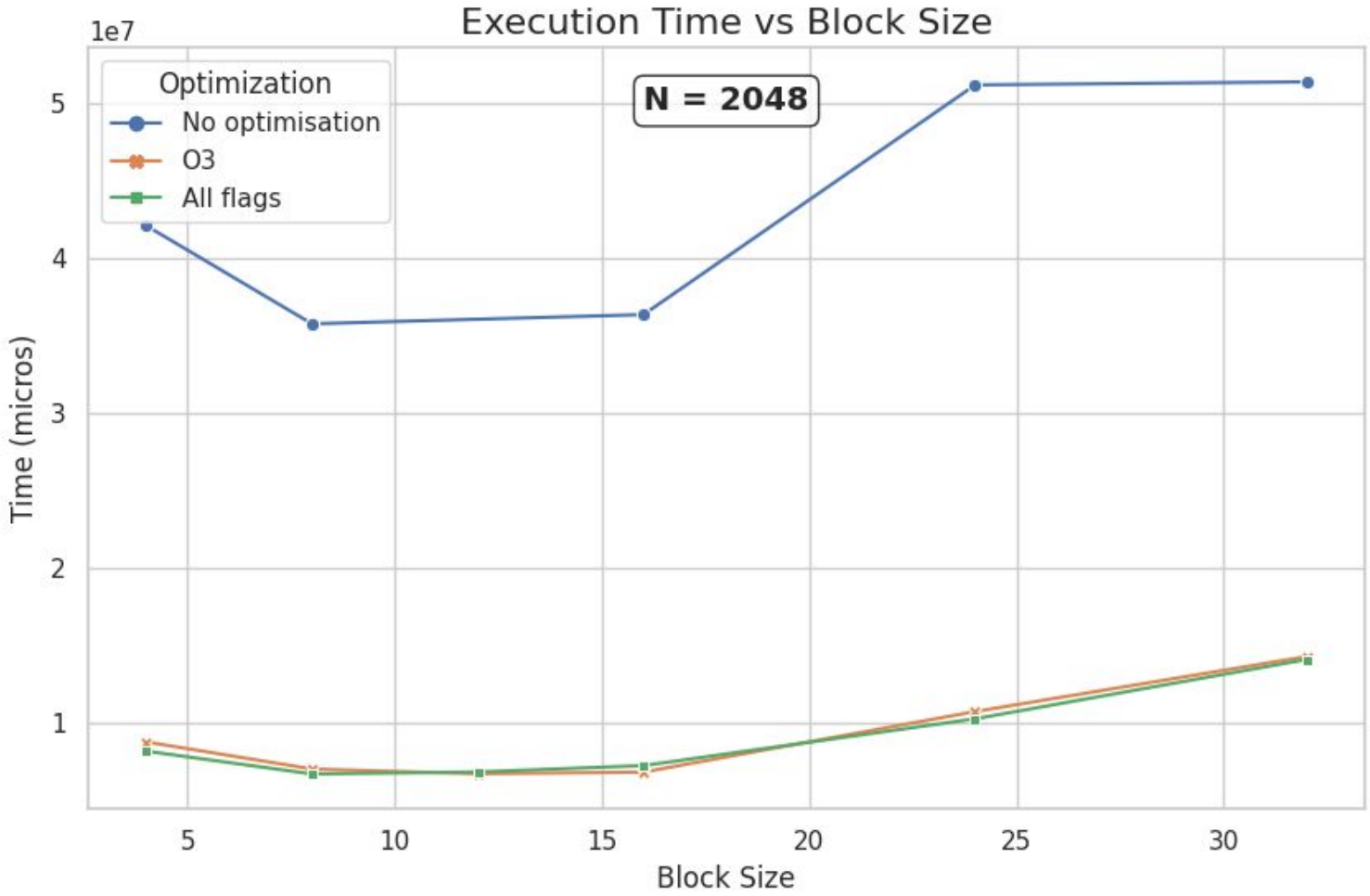
Use vector registers

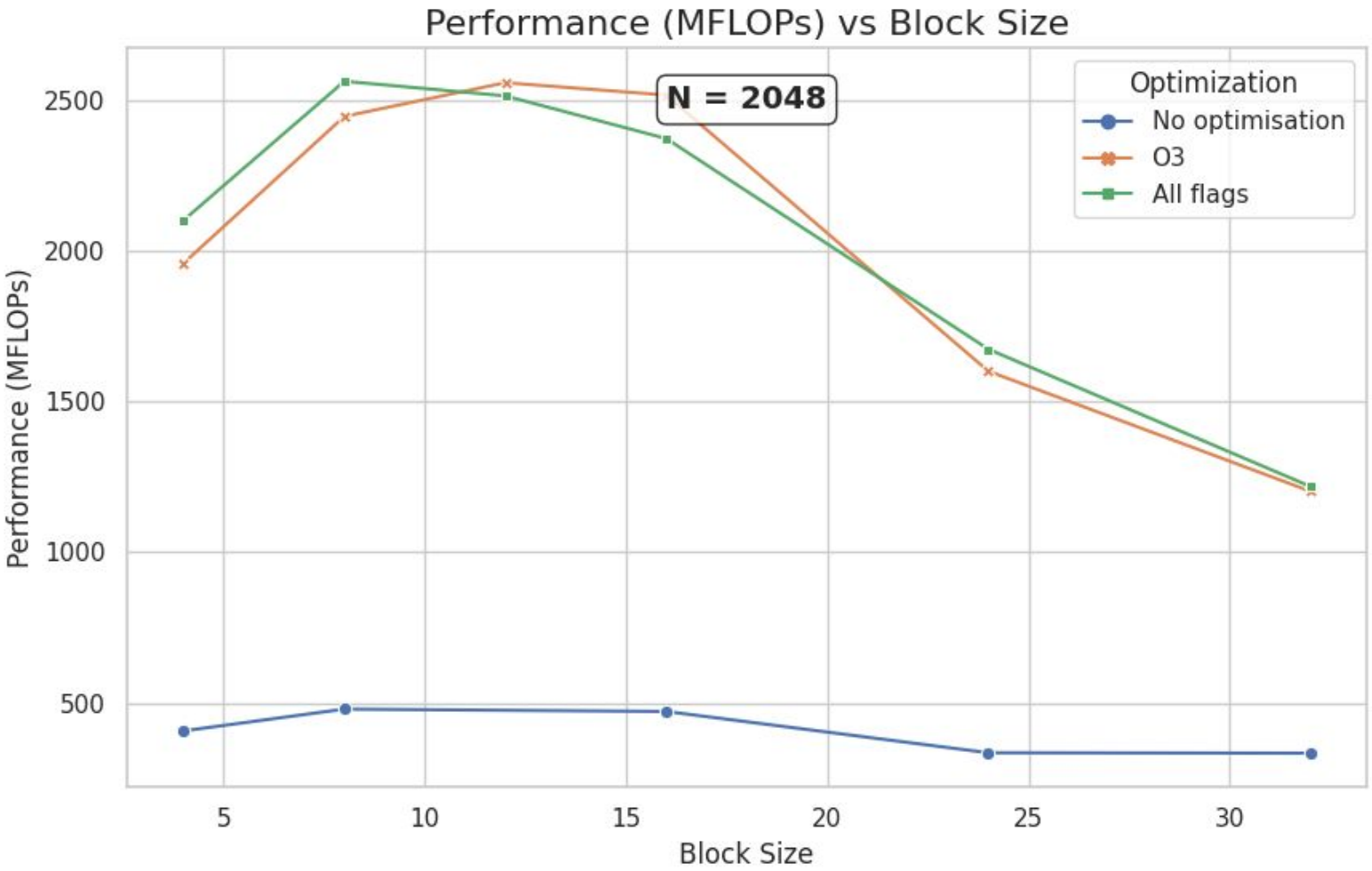
```
Standard -O3 (Old)                  FMA Optimized (New)
-------------------------------     ---------------------------------
.L10:                               .L10:
 movsd   (%rax), %xmm1               vmovsd  (%rax), %xmm1            ; Load A

 mulsd   (%rdx), %xmm1               (Merged into next line)          ; Multiply
 addsd   %xmm1, %xmm0                vfmadd231sd (%rdx), %xmm1, %xmm0 ; Mult + Add

 addq    $8, %rax                    addq    $8, %rax                 ; Incr A ptr
 addq    %rcx, %rdx                  addq    %rcx, %rdx               ; Incr B ptr
 cmpq    %rsi, %rax                  cmpq    %rax, %rsi               ; Check loop
 jne     .L10                        jne     .L10                     ; Repeat
```

```
movapd  %xmm0, %xmm3                    |       vaddsd  %xmm0, %xmm0, %xmm3

addsd   %xmm2, %xmm2                    |       vaddsd  %xmm2, %xmm2, %xmm2

addq    $8, %rax                                addq    $8, %rax
                                    >           vfmadd231sd     (%rdx), %xmm1, %xmm0

addq    $8, %rax                                        addq    $8, %rax
                                            >           vfmadd231sd     (%rdx), %xmm1, %xmm0
```

Check relative performance

User	size_KB	BlockSize	BlocSize_KB	time_micros	Performance_MFLOPs	Optimization
6221	98304.0	4	0.38	42105297.00	408.02	No optimisation
6221	98304.0	8	1.50	35766768.67	480.33	No optimisation
6221	98304.0	16	6.00	36359278.33	472.50	No optimisation
6221	98304.0	24	13.50	51186910.33	335.63	No optimisation
6221	98304.0	32	24.00	51387338.67	334.32	No optimisation
6221	98304.0	4	0.38	8774653.67	1957.90	O3
6221	98304.0	8	1.50	7026139.33	2445.14	O3
6221	98304.0	12	3.38	6719190.00	2556.84	O3
6221	98304.0	16	6.00	6829173.33	2515.66	O3
6221	98304.0	24	13.50	10734199.67	1600.48	O3
6221	98304.0	32	24.00	14287151.00	1202.47	O3
6221	98304.0	48	54.00	16185334.33	1061.45	O3
6221	98304.0	4	0.38	8179341.33	2100.40	All flags
6221	98304.0	8	1.50	6706457.33	2561.69	All flags
6221	98304.0	12	3.38	6836296.00	2513.04	All flags
6221	98304.0	16	6.00	7244410.33	2371.47	All flags
6221	98304.0	24	13.50	10266524.33	1673.39	All flags
6221	98304.0	32	24.00	14098463.00	1218.56	All flags


Execution Time vs Block Size
1e7
Optimization
No optimisation
O3
All flags
N = 2048
Time (micros)
Block Size
1
2
3
4
5
5
10
15
20
25
30


Performance (MFLOPs) vs Block Size
N = 2048
Optimization
No optimisation
O3
All flags
Performance (MFLOPs)
2500
2000
1500
1000
500
5
10
15
20
25
30
Block Size

Write memory aware code.

```c
#if 1
void blocked_gemm(int N, double * restrict A, double * restrict B, double * restrict C, int BLOCK_SIZE) {
#pragma omp parallel for collapse(2)
    for (int i = 0; i < N; i += BLOCK_SIZE) {
        for (int k = 0; k < N; k += BLOCK_SIZE) {
            for (int j = 0; j < N; j += BLOCK_SIZE) {
                // Compute block multiplication
                for (int ii = i; ii < i + BLOCK_SIZE && ii < N; ++ii) {
                    for (int kk = k; kk < k + BLOCK_SIZE && kk < N; ++kk) {
                        double a_val = A[ii * N + kk];
                        for (int jj = j; jj < j + BLOCK_SIZE && jj < N; ++jj) {
                            C[ii * N + jj] += a_val * B[kk * N + jj];
                        }
                    }
                }
            }
        }
    }
}
```

User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	Performance(MFLOPs)
6221	node02.cluster.taz	2048	98304.00	8	1.50	4993131.33	3440.70

```
[psingh@coka logs]$ cat serial_nop_ps_2133772.err

Performance counter stats for './gemm-x86_64.x 2048 8':

         14,972.62 msec task-clock                #    0.999 CPUs utilized
               796      context-switches          #    0.053 K/sec
                 1      cpu-migrations            #    0.000 K/sec
             3,349      page-faults               #    0.224 K/sec
  46,40,32,11,569      cycles                    #    3.099 GHz
  85,67,90,84,106      instructions              #    1.85  insn per cycle
   9,77,63,80,526      branches                  #  652.951 M/sec
         6,69,217      branch-misses             #    0.01% of all branches
  38,52,49,64,768      L1-dcache-loads           # 2573.028 M/sec
   4,58,66,74,804      L1-dcache-load-misses     #   11.91% of all L1-dcache hits
       3,22,88,185      LLC-loads                 #    2.156 M/sec
            88,699      LLC-load-misses           #    0.27% of all LL-cache hits

      14.982190941 seconds time elapsed

      14.961852000 seconds user
       0.012998000 seconds sys
```

```
#else
void blocked_gemm(int N, double * restrict A, double * restrict B, double * restrict C, int BLOCK_SIZE) {
    #pragma omp parallel for collapse(2)
    for (int i = 0; i < N; i += BLOCK_SIZE) {
        for (int j = 0; j < N; j += BLOCK_SIZE) {
            for (int k = 0; k < N; k += BLOCK_SIZE) {
                for (int ii = i; ii < i + BLOCK_SIZE && ii < N; ++ii) {
                    for (int jj = j; jj < j + BLOCK_SIZE && jj < N; ++jj) {
                        double sum = C[ii * N + jj];
                        for (int kk = k; kk < k + BLOCK_SIZE && kk < N; ++kk) {
                            sum += A[ii * N + kk] * B[kk * N + jj];
                        }
                        C[ii * N + jj] = sum;
                    }
                }
            }
        }
    }
}
#endif
```

User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	Performance(MFLOPs)
6221	node02.cluster.taz	2048	98304.00	8	1.50	6706560.67	2561.65

```
[psingh@coka logs]$ cat serial_nop_ps_2133777.err

 Performance counter stats for './gemm-x86_64.x 2048 8':

         20,095.58 msec task-clock                #    0.999 CPUs utilized
             1,261      context-switches          #    0.063 K/sec
                 3      cpu-migrations            #    0.000 K/sec
             3,326      page-faults               #    0.166 K/sec
  62,39,48,75,691      cycles                    #    3.105 GHz
1,95,21,72,76,100      instructions              #    3.13  insn per cycle
  32,68,11,98,091      branches                  # 1626.288 M/sec
     5,09,65,759      branch-misses             #    0.16% of all branches
  55,54,25,77,223      L1-dcache-loads           # 2763.920 M/sec
   5,06,67,76,556      L1-dcache-load-misses     #    9.12% of all L1-dcache hits
     80,32,55,727      LLC-loads                 #   39.972 M/sec
      5,97,291      LLC-load-misses           #    0.07% of all LL-cache hits

    20.122084468 seconds time elapsed

    20.089335000 seconds user
     0.009999000 seconds sys
```

Scope of parallelisation

```
gemm: gemm.c
	gcc -O3 -fopenmp -mfma -march=native  -mtune=native  $^ -o $@-$(arch).x
```

```
#include <string.h>
#include <stdint.h>
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/time.h>
#include <omp.h>
```

```
#!/bin/bash
#SBATCH --partition=shortrun
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=16
#SBATCH --time=00:10:00
#SBATCH --job-name=scaling_test
#SBATCH --hint=nomultithread
#SBATCH --output=logs/omp_mat_%j.out
#SBATCH --error=logs/omp_mat_%j.err
##SBATCH --hint=compute_bound
#SBATCH --cpu-freq=3000000
#SBATCH --exclusive
```

```
void blocked_gemm(int N, double * restrict A, double * restrict B, double * restrict C, int BLOCK_SIZE) {
#pragma omp parallel for collapse(2)
```

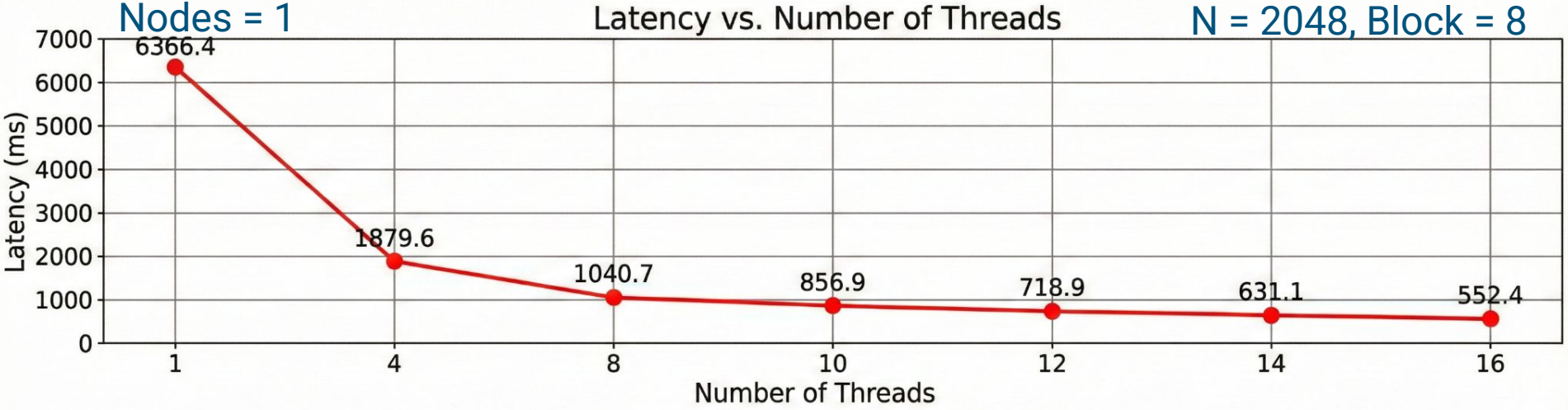
```
OPENMP DISPLAY ENVIRONMENT BEGIN
  _OPENMP = '201511'
  OMP_DYNAMIC = 'FALSE'
  OMP_NESTED = 'FALSE'
  OMP_NUM_THREADS = '8'
  OMP_SCHEDULE = 'DYNAMIC'
  OMP_PROC_BIND = 'CLOSE'
  OMP_PLACES = '{0},{1},{2},{3},{4},{5},{6},{7}'
  OMP_STACKSIZE = '140722303540382'
  OMP_WAIT_POLICY = 'PASSIVE'
  OMP_THREAD_LIMIT = '4294967295'
  OMP_MAX_ACTIVE_LEVELS = '2147483647'
  OMP_CANCELLATION = 'FALSE'
  OMP_DEFAULT_DEVICE = '0'
  OMP_MAX_TASK_PRIORITY = '0'
OPENMP DISPLAY ENVIRONMENT END
```

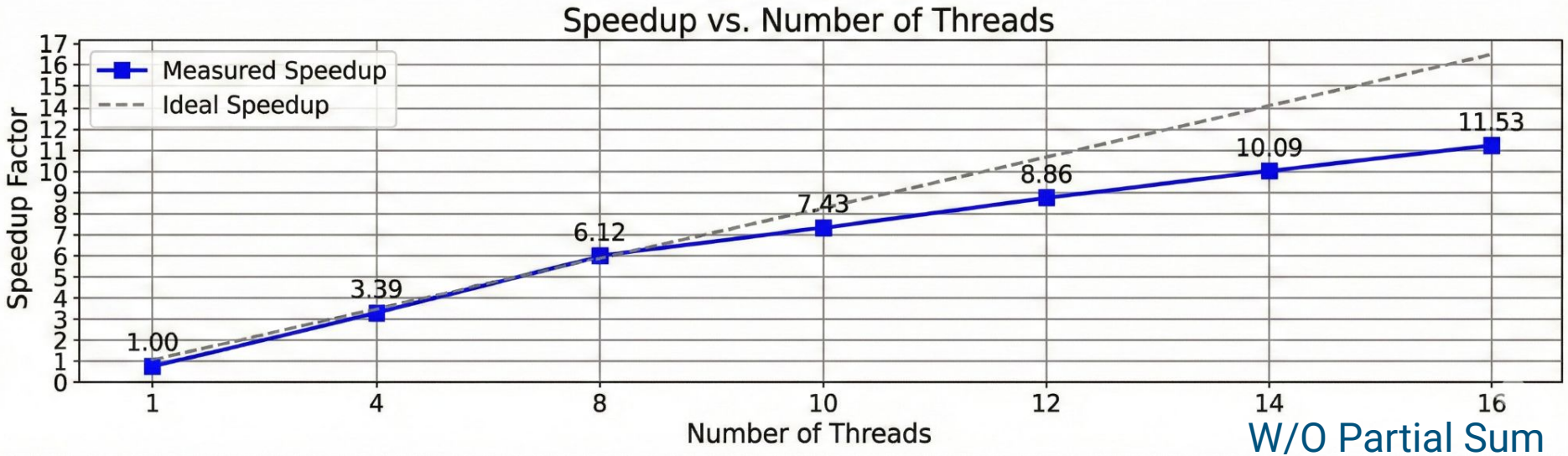
* OMP Threads <= 8

* 8 <= OMP Threads <= 16

```
OPENMP DISPLAY ENVIRONMENT BEGIN
  _OPENMP = '201511'
  OMP_DYNAMIC = 'FALSE'
  OMP_NESTED = 'FALSE'
  OMP_NUM_THREADS = '10'
  OMP_SCHEDULE = 'DYNAMIC'
  OMP_PROC_BIND = 'CLOSE'
  OMP_PLACES = '{0},{1},{2},{3},{4},{5},{6},{7},{8},{9},{10},
                {11},{12},{13},{14},{15}'
  OMP_STACKSIZE = '140724784442526'
  OMP_WAIT_POLICY = 'PASSIVE'
  OMP_THREAD_LIMIT = '4294967295'
  OMP_MAX_ACTIVE_LEVELS = '2147483647'
  OMP_CANCELLATION = 'FALSE'
  OMP_DEFAULT_DEVICE = '0'
  OMP_MAX_TASK_PRIORITY = '0'
OPENMP DISPLAY ENVIRONMENT END
```

Nodes = 1

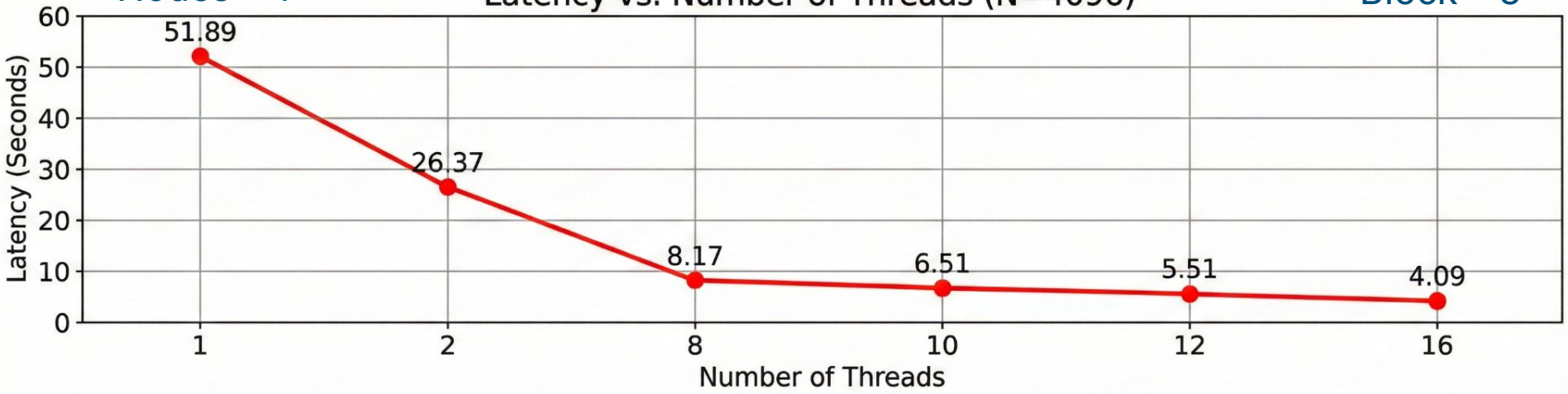


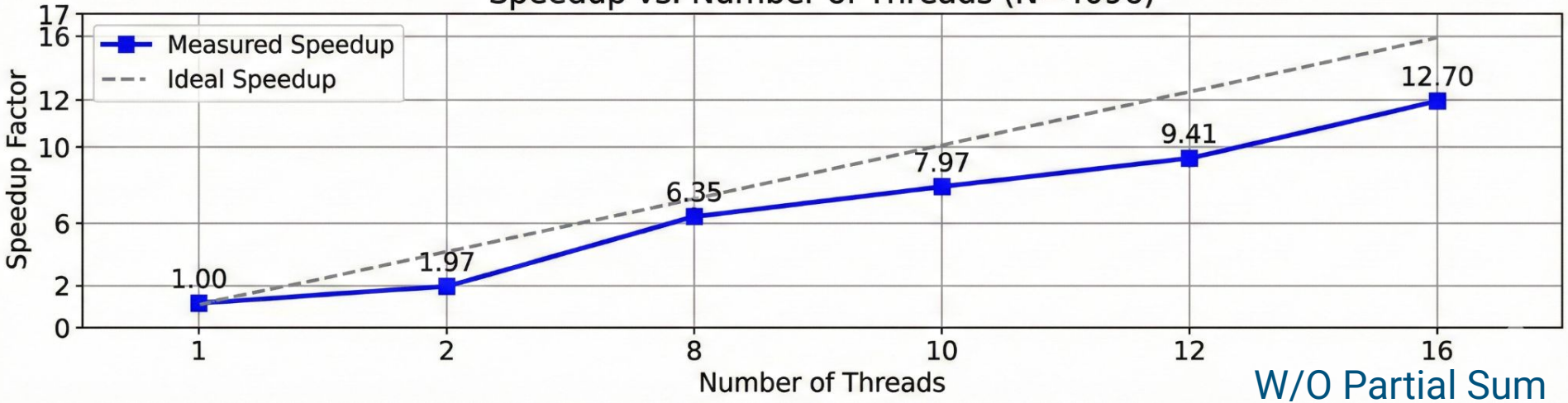


W/O Partial Sum

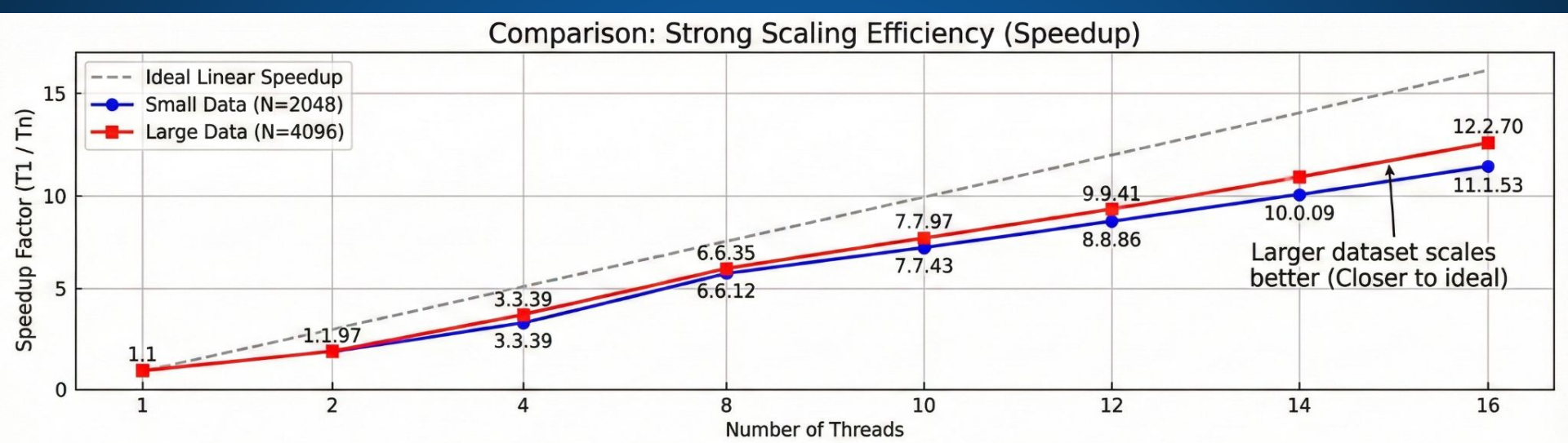
Nodes = 1

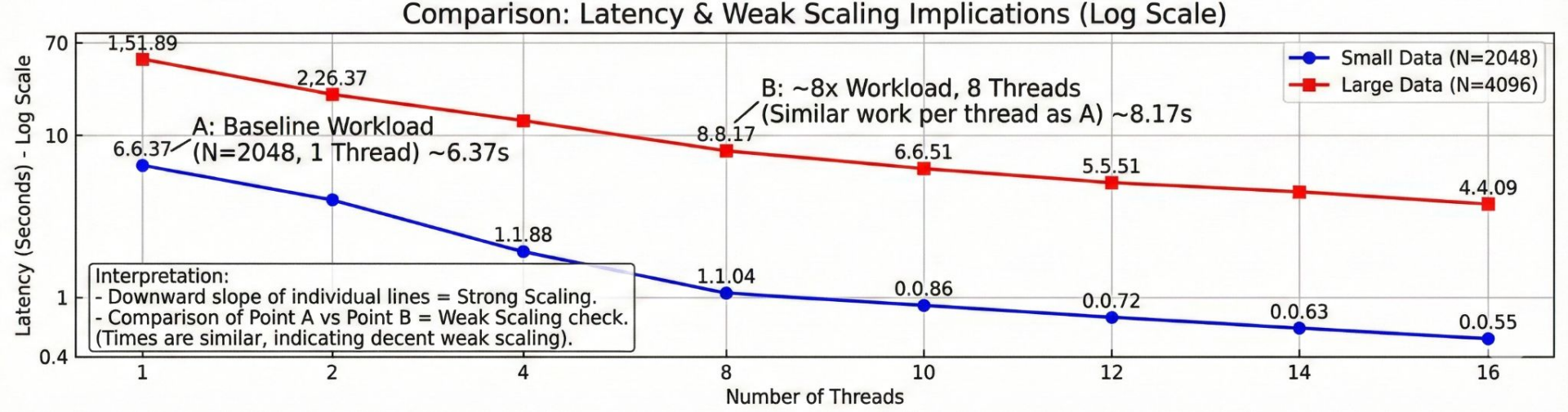
Block = 8





W/O Partial Sum


Comparison: Strong Scaling Efficiency (Speedup)
Ideal Linear Speedup
Small Data (N=2048)
Large Data (N=4096)
Speedup Factor (T1 / Tn)
Number of Threads
1.1
1.1.97
3.3.39
3.3.39
6.6.35
6.6.12
7.7.97
7.7.43
9.9.41
8.8.86
10.0.09
12.2.70
11.1.53
Larger dataset scales
better (Closer to ideal)


Comparison: Latency & Weak Scaling Implications (Log Scale)
Small Data (N=2048)
Large Data (N=4096)
Latency (Seconds) - Log Scale
Number of Threads
1,51.89
2,26.37
8.8.17
6.6.51
5.5.51
4.4.09
6.6.37
1.1.88
1.1.04
0.0.86
0.0.72
0.0.63
0.0.55
A: Baseline Workload
(N=2048, 1 Thread) ~6.37s
B: ~8x Workload, 8 Threads
(Similar work per thread as A) ~8.17s
Interpretation:
- Downward slope of individual lines = Strong Scaling.
- Comparison of Point A vs Point B = Weak Scaling check.
(Times are similar, indicating decent weak scaling).

Comparing Arithmetic Intensity

Without partial sum, optimised by compiler

User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	MFLOPs	Bandwidth(GB/s)
6221	node02.cluster.taz	2048	98304.00	8	1.50	5235253.67	3281.57	6.11

With partial sum, optimised by compiler

*Serial Execution

User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	MFLOPs	Bandwidth(GB/s)
6221	node02.cluster.taz	2048	98304.00	8	1.50	9712798.67	1768.79	3.29

With partial sum, optimised by compiler, omp threads = 16

User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	MFLOPs	Bandwidth(GB/s)
6221	node02.cluster.taz	2048	98304.00	8	1.50	6398683.33	2684.91	5.00

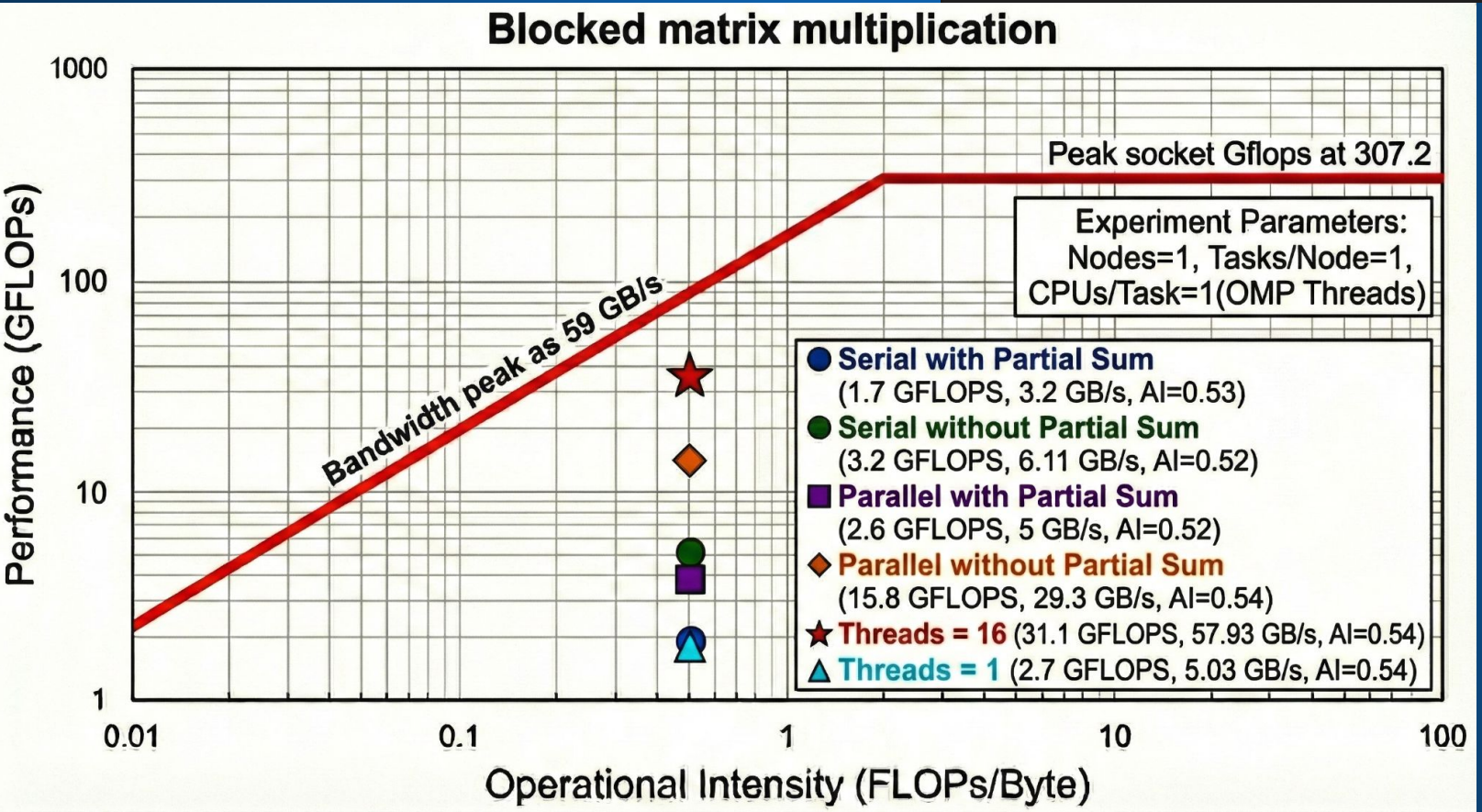
Without partial sum, optimised by compiler, omp threads = 16

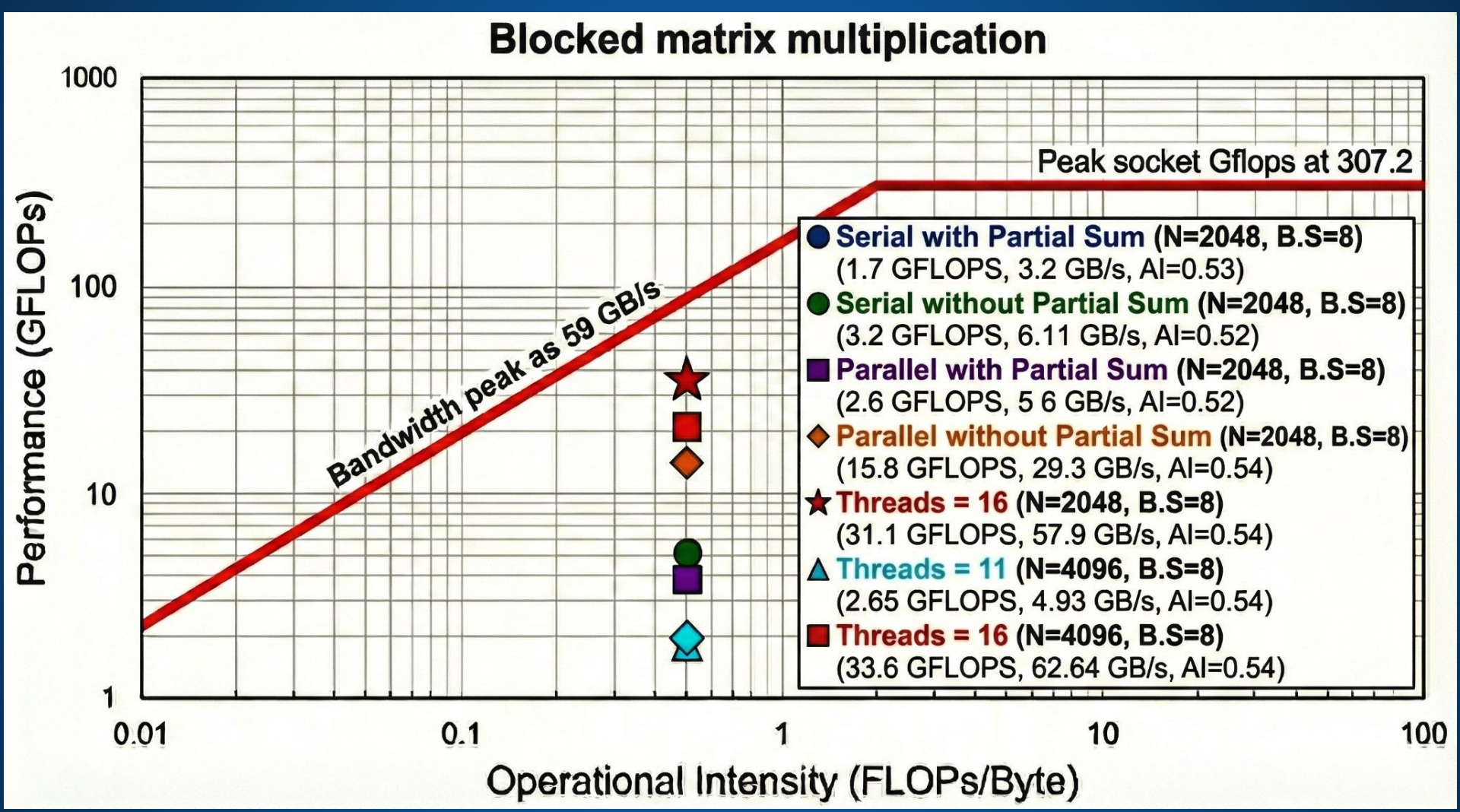
User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	MFLOPs	Bandwidth(GB/s)
6221	node01.cluster.taz	2048	98304.00	8	1.50	1083598.67	15854.46	29.53

$$I = Flops/Bytes$$

Roofline Model

```
Machine Balance = Flops (Peak) / Bandwidth (Peak)
                = (16 * 2.4 * 1e9 * 8) / (59 * 1e9)
                = 307.2 * 1e9 / 59 * 1e9
                = 5.206
```




Blocked matrix multiplication
1000
100
10
1
Performance (GFLOPs)
0.01
0.1
1
10
100
Operational Intensity (FLOPs/Byte)
Peak socket Gflops at 307.2
Bandwidth peak as 59 GB/s
Serial with Partial Sum (N=2048, B.S=8)
(1.7 GFLOPS, 3.2 GB/s, AI=0.53)
Serial without Partial Sum (N=2048, B.S=8)
(3.2 GFLOPS, 6.11 GB/s, AI=0.52)
Parallel with Partial Sum (N=2048, B.S=8)
(2.6 GFLOPS, 5 6 GB/s, AI=0.52)
Parallel without Partial Sum (N=2048, B.S=8)
(15.8 GFLOPS, 29.3 GB/s, AI=0.54)
Threads = 16 (N=2048, B.S=8)
(31.1 GFLOPS, 57.9 GB/s, AI=0.54)
Threads = 11 (N=4096, B.S=8)
(2.65 GFLOPS, 4.93 GB/s, AI=0.54)
Threads = 16 (N=4096, B.S=8)
(33.6 GFLOPS, 62.64 GB/s, AI=0.54)

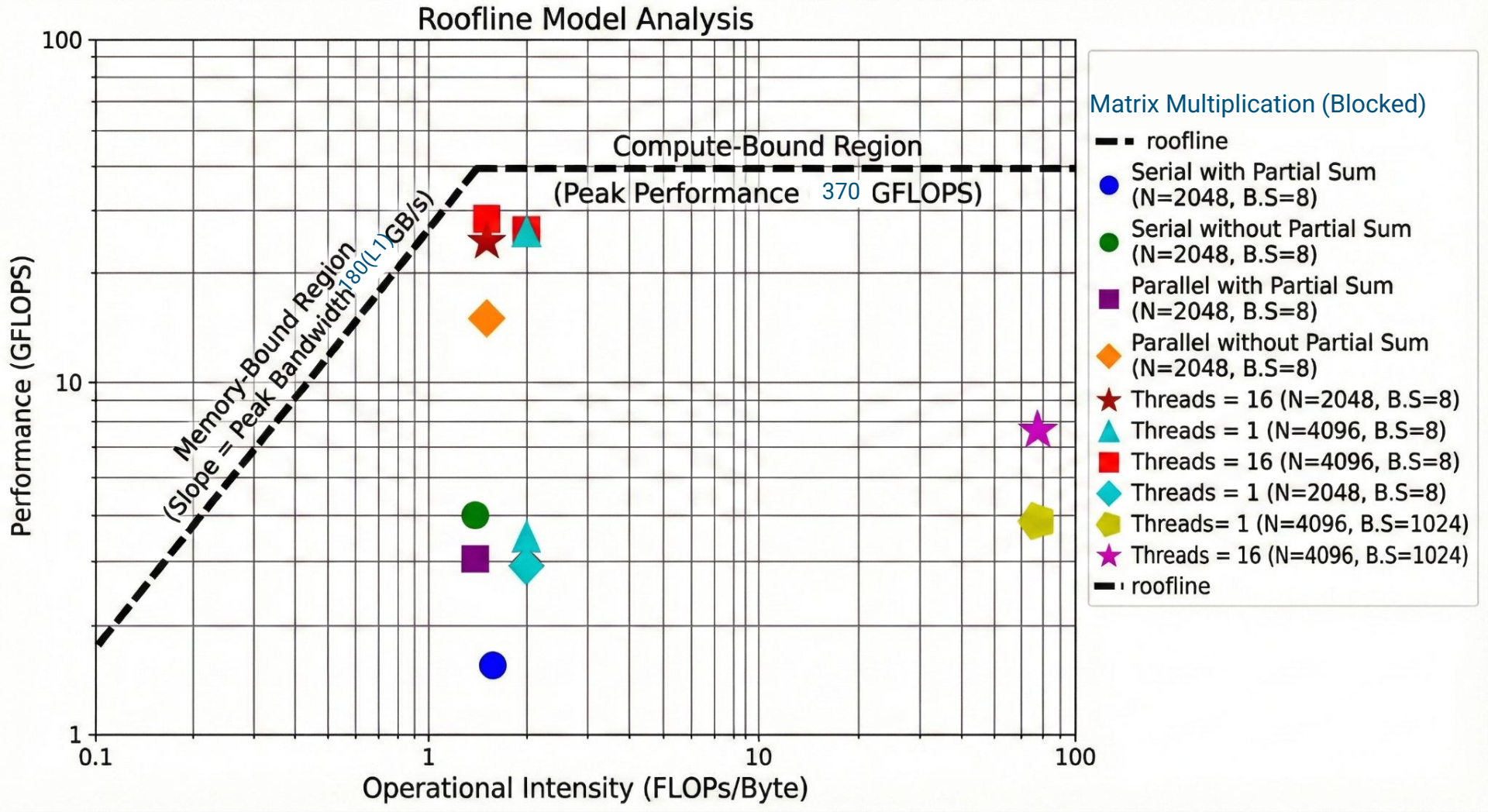
Optimise to the bone.

```
gemm: gemm.c
	gcc -O3  $^ -o $@-$(arch).x
```

User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	Performance(MFLOPs/s)	Bandwidth(GBytes/s)
6221	node01.cluster.taz	2048	98304.00	4	0.38	9562003.00	1796.68	6.69
6221	node01.cluster.taz	2048	98304.00	8	1.50	6269975.33	2740.02	5.10
6221	node01.cluster.taz	2048	98304.00	12	3.38	5390619.00	3186.99	3.96
6221	node01.cluster.taz	2048	98304.00	16	6.00	4668155.67	3680.23	3.43
6221	node01.cluster.taz	2048	98304.00	24	13.50	4778072.00	3595.57	2.23
6221	node01.cluster.taz	2048	98304.00	32	24.00	4459901.00	3852.07	1.79
6221	node01.cluster.taz	2048	98304.00	48	54.00	3813839.67	4504.61	1.40
6221	node01.cluster.taz	2048	98304.00	64	96.00	3678672.00	4670.13	1.09
6221	node01.cluster.taz	2048	98304.00	96	216.00	3454294.67	4973.48	0.77
6221	node01.cluster.taz	2048	98304.00	128	384.00	3176238.00	5408.87	0.63
6221	node01.cluster.taz	2048	98304.00	192	864.00	2939631.00	5844.23	0.45
6221	node01.cluster.taz	2048	98304.00	256	1536.00	2983014.00	5759.23	0.34
6221	node01.cluster.taz	2048	98304.00	384	3456.00	2804891.67	6124.97	0.24
6221	node01.cluster.taz	2048	98304.00	512	6144.00	2788220.00	6161.59	0.18
6221	node01.cluster.taz	2048	98304.00	768	13824.00	2690161.00	6386.19	0.12
6221	node01.cluster.taz	2048	98304.00	1024	24576.00	2615208.33	6569.22	0.10
6221	node01.cluster.taz	2048	98304.00	1536	55296.00	4116193.33	4173.73	0.04
6221	node01.cluster.taz	2048	98304.00	2048	98304.00	5201126.67	3303.11	0.02

```
gemm: gemm.c
	gcc -O3 -march=native  -mtune=native -mfma  $^ -o $@-$(arch).x
```

User	Machine	size(N)	size(KB)	BlockSize	BlocSize(KB)	time(micros)	Performance(MFLOPs/s)	Bandwidth(GBytes/s)
6221	node01.cluster.taz	2048	98304.00	4	0.38	8451623.67	2032.73	7.57
6221	node01.cluster.taz	2048	98304.00	8	1.50	5333064.33	3221.39	6.00
6221	node01.cluster.taz	2048	98304.00	12	3.38	4489479.00	3826.70	4.75
6221	node01.cluster.taz	2048	98304.00	16	6.00	3762040.67	4566.64	4.25
6221	node01.cluster.taz	2048	98304.00	24	13.50	4759438.00	3609.64	2.24
6221	node01.cluster.taz	2048	98304.00	32	24.00	4668329.33	3680.09	1.71
6221	node01.cluster.taz	2048	98304.00	48	54.00	4279888.67	4014.09	1.25
6221	node01.cluster.taz	2048	98304.00	64	96.00	3840000.00	4473.92	1.04
6221	node01.cluster.taz	2048	98304.00	96	216.00	3550620.33	4838.55	0.75
6221	node01.cluster.taz	2048	98304.00	128	384.00	3205831.33	5358.94	0.62
6221	node01.cluster.taz	2048	98304.00	192	864.00	2952998.33	5817.77	0.45
6221	node01.cluster.taz	2048	98304.00	256	1536.00	2789288.67	6159.23	0.36
6221	node01.cluster.taz	2048	98304.00	384	3456.00	2739116.00	6272.05	0.24
6221	node01.cluster.taz	2048	98304.00	512	6144.00	2666449.33	6442.98	0.19
6221	node01.cluster.taz	2048	98304.00	768	13824.00	2556734.67	6719.46	0.13
6221	node01.cluster.taz	2048	98304.00	1024	24576.00	2381078.00	7215.16	0.10
6221	node01.cluster.taz	2048	98304.00	1536	55296.00	3629585.33	4733.29	0.05


Roofline Model Analysis
Compute-Bound Region
(Peak Performance 370 GFLOPS)
Memory-Bound Region
(Slope = Peak Bandwidth 180(L1) GB/s)
Performance (GFLOPS)
Operational Intensity (FLOPs/Byte)
100
10
1
0.1
1
10
100
Matrix Multiplication (Blocked)
roofline
Serial with Partial Sum (N=2048, B.S=8)
Serial without Partial Sum (N=2048, B.S=8)
Parallel with Partial Sum (N=2048, B.S=8)
Parallel without Partial Sum (N=2048, B.S=8)
Threads = 16 (N=2048, B.S=8)
Threads = 1 (N=4096, B.S=8)
Threads = 16 (N=4096, B.S=8)
Threads = 1 (N=2048, B.S=8)
Threads= 1 (N=4096, B.S=1024)
Threads = 16 (N=4096, B.S=1024)
roofline

Issues: Diff in slurm configs

```
#!/bin/bash
#SBATCH --partition=shortrun
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=16
#SBATCH --time=00:10:00
#SBATCH --job-name=scaling_test
#SBATCH --hint=nomultithread
#SBATCH --output=logs/%x_%j.out
#SBATCH --error=logs/%x_%j.err
##SBATCH --hint=compute_bound
#SBATCH --cpu-freq=3000000
#SBATCH --exclusive

mkdir -p logs
```

```
scontrol: show job 2133548
JobId=2133548 JobName=scaling_test
   UserId=psingh(6221) GroupId=cusers(101)
   Priority=732 Nice=0 Account=fst QOS=normal
   JobState=FAILED Reason=NonZeroExitCode Dependency=(null)
   Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=1:0
   RunTime=00:00:00 TimeLimit=00:10:00 TimeMin=N/A
   SubmitTime=2025-12-11T14:04:16 EligibleTime=2025-12-11T14:04:16
   StartTime=2025-12-11T14:04:17 EndTime=2025-12-11T14:04:17
   PreemptTime=None SuspendTime=None SecsPreSuspend=0
   Partition=shortrun AllocNode:Sid=coka:17949
   ReqNodeList=(null) ExcNodeList=(null)
   NodeList=node02
   BatchHost=node02
   NumNodes=1 NumCPUs=16 CPUs/Task=16 ReqB:S:C:T=0:0:*:1
   TRES=cpu=16,mem=16000,node=1
   Socks/Node=* NtasksPerN:B:S:C=1:0:*:1 CoreSpec=*
   MinCPUsNode=16 MinMemoryCPU=1000M MinTmpDiskNode=0
   Features=(null) Gres=(null) Reservation=(null)
   Shared=0 Contiguous=0 Licenses=(null) Network=(null)
   Command=/home/psingh/P1.9_arch_2025/mhpc-codes/gemm-block/batch.sh
   WorkDir=/home/psingh/P1.9_arch_2025/mhpc-codes/gemm-block
   StdErr=/home/psingh/P1.9_arch_2025/mhpc-codes/gemm-block/logs/%x_2133548.err
   StdIn=/dev/null
   StdOut=/home/psingh/P1.9_arch_2025/mhpc-codes/gemm-block/logs/%x_2133548.out
   CPU_max_freq=3000000
   Power= SICP=0

scontrol:
```

Thanks.

Some Q&A perhaps.